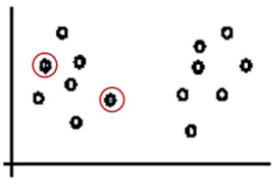
- 2 approcci da sapere così:
  - Ridurre della dimensionalità (quali attributi si possono non considerare)
  - Detect delle anomalie (Usato per i bug/crash)
- Clustering
  - Creiamo in automatico dei gruppi sulla base di una similarità
    - Istanze che sono simili date delle features data una funzione
    - Spesso il clustering è più esplorazione che classificazione
      - Siccome serve per trovare relazioni geometriche
      - E se si trovano clustering sensati, allora potremmo **poi** fare una classificazione
  - Non lavora su dati non etichettati e nemmeno il numero dei gruppi (a volte può essere specificato)
    - Ci serve una funzione che ci dice quanto 2 punti sono distanti
  - I cluster si evalutano attraverso 2 metriche
    - Distanza intra-cluster
      - Coesione del cluster
      - Quanto i punti sono vicini
      - Deve essere minimizzato
        - Aka la densità
    - Distanza inter-cluster
      - Vogliamo 2 cluster il più distanti possibile
      - Massimizzare
  - Diverse tipologie
    - Partizionali
      - Esempio: k-means
        - Dati un insieme di n vettori di attributi
        - usiamo la media per creare k cluster
          - K deve essere specificato dall'utente
        - Il risultato sarà
          - k cluster ognuno caratterizzato da un centroide (il punto che sta al centro del cluster)
          - k-means riesce a classificare cluster che sono **solamente** sferiche
        - Come si valutano:
          - E' difficile, spesso è un utente a farlo
          - Ground truth: noi facciamo la classificazione parziale a mano, e da li possiamo fare una confusion matrix

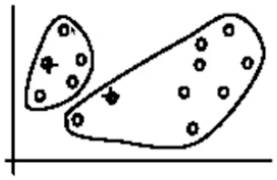
        - E' anche utile vedere come gli utenti reagiscono a ciò che noi suggeriamo
          Es. suggerisco un film, e se li piacerà clustering andava bene, senò no
        - E nota, anche se abbiamo una piccola parte di etichette, non sappiamo se farà errori da altre parti
    - Oppure i generici
        - Coesione intra cluster: la coesione dei cluster, fatto con la somma degli errori quadrati
        - Inter-cluster separation: quanti i cluster sono lontani
- Algoritmo
    1. Dato un dataset, prendiamo k dati a caso e decidiamo la loro posizione come k centroidi
    2. Poi prendiamo tutti i dati, e li associamo al centroide più vicino
    3. Ricalcoliamo i centroidi
    4. E se il criterio di convergenza non è raggiunto, torniamo punto 2
       Ce ne sono vari di criteri
        - Il movimento dei centroidi durante la ricalcolazione è minima
        - Minimo riassegnamento di dati a cluster differenti (aka nessun dato si sta più spostando di cluster)
        - Ho trovato dei cluster abbastanza densi/compatti, che si calcola come la somma della distanza di tutti i dati dal suo centroidi, al quadrato

$$SSE = \sum_{j=1}^{k} \sum_{\mathbf{x} \in C_j} dist(\mathbf{x}, \mathbf{m}_j)^2$$

- Esempio:



(A). Random selection of k centers



Iteration 1: (B). Cluster assignment

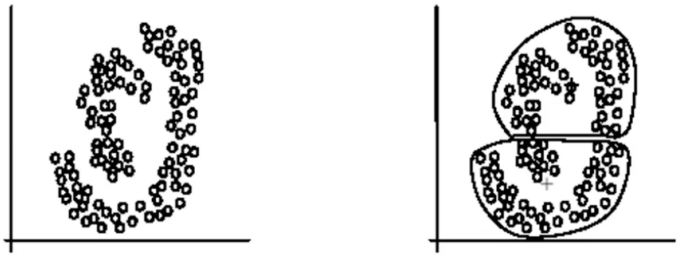


(C). Re-compute centroids

- I cerchi in alto rosso sono i 2 centroidi selezionati che corrispondono a 2 punti casuali

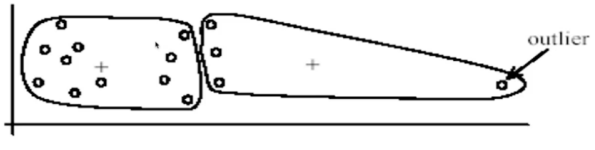
- E notiamo come in C, il punto medio si è spostato (lo vedi dal +)
- Poi si continua così
- Spesso la funzione di distanza è la tipica distanza euclidea

- Forza:
  - Algoritmo semplice
  - Efficente in complessità
  - E' un algoritmo lineare
- Debolezze:
  - Ci potremmo trovare in ottimili locali bellissimi, la compettezza è soddisfatta, però non è un ottimo globale
    - Soluzione: si rilancia kmeans
    - E' possibile che 2 kmeans uguali su stesso dataset da risultati diversi
  - Non riusciamo a gestire cluster che non sono delle sfere

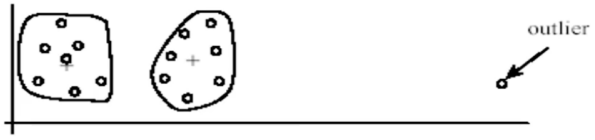


(A): Two natural clusters (B): *k*-means clusters

  - La k deve venire specificata
    - Si va a tentativi senò finchè non troviamo k migliore
  - Funziona solo su attributi numerici (ameno che non troviamo una funzione di distanza noi)
  - Come si gestiscono gli outliners
    - Punti geometricamente molto lontani



(A): Undesirable clusters



(B): Ideal clusters

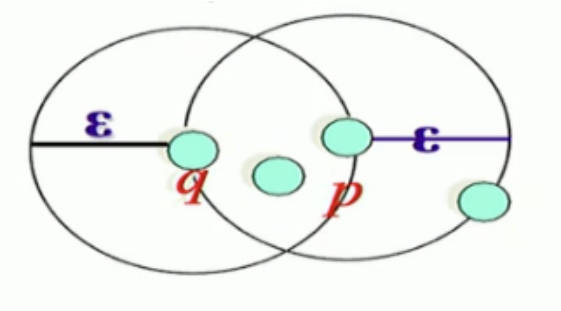
soluzione:

    - Ogni volta che creiamo i nuovi cluster, se ci sono 1 o 2 punti troppo lontani rispetto agli altri, lo ignoriamo/rimuoviamo

Va bene se è un outliner, però non se è un informazione interessante (top spender)
- Random samping: prendiamo dei sottoinsiemi randomici
  Siccome gli outliner sono pochi, statisticamente è difficile che si trovino se è casuale
  E poi assegneremo gli outliner al centroide più vicino

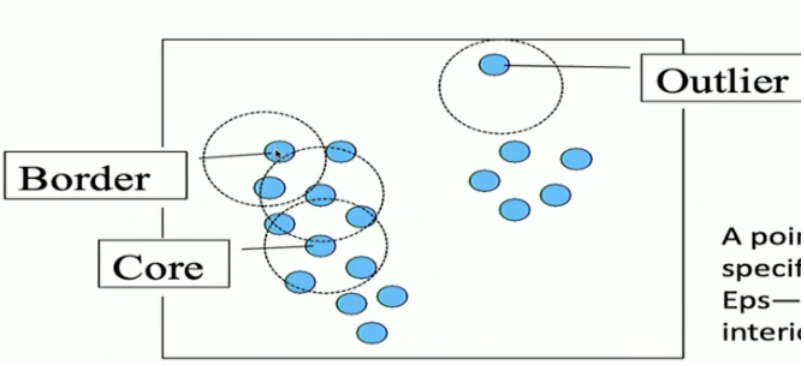
- Density-based clustering (DBScan)
  - Concetto: punti che stanno vicino, densi, vanno a creare cluster. La divisione dei cluster è dove ci sono o un pò di punti o nessun punto
  - E' possibile trovare una qualunque forma qui

- Gerarchico
  - Possono creare cluster dentro cluster
  - Qui è l'algoritmo che decide quanti cluster, non l'utente
  - Forza:
    - Può identificare forme di classi complicate
    - Non ha complessità esagerata
  - Debolezza:
    - Molto sensibile ai parametri
      Soluzione:
      - Faccio una stima con un k-means
      - Poi metto il risultato in un grafo con la media della distanza per punti, e da qui l'utente deciderà
    - Con piccoli offset di troppo nei dati potrebbe creare però 2 cluster diversi quando non di dovrebbero essere
      - Aka, ha difficoltà a individuare dataset con densità molto variabile
  - Algoritmo
    - Tutto è basato sull'epsilon vicinato
      - Prendiamo un punto, da questo disegniamo una circonferenza di raggio epsilon, tutti gli altri punti dentro fanno parte dell'epsilon vicinato
        - **Epsilon** è un parametro che noi li diamo
      - Poi dato un parametro **minpoints**, contiamo i punti dell'epsilon vicinato
        - Se abbiamo più di minpoints allora ha una densità molto alta
        - Senò si trova in un aria di bassa densità
          Assumendo minpoints=4, p è in alta densità, ed invece q è di bassa densità

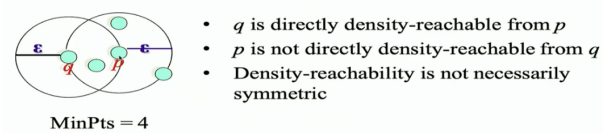


E da questo possiamo classificare i punti in:

- Core point, punto con più di minpoints (alta densità, dentro al cluster)
- Border point, punti che hanno meno di min points però appartengono ad almeno 1 punto core
- Noise/Outliner, sono punti che non sono core

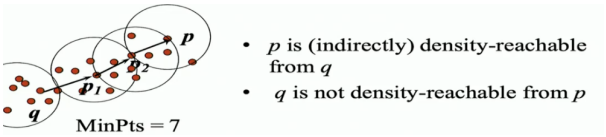


- E dopo avere etichettato, si fa la raggiungibilità tramite densità
  Lega 2 punti p e q se p è un punto core mentre q è nel suo epsilon avvicinato



  Qui io posso raggiungere p direttamente da q (relazione non simmetrica siccome q **non è** punto core)

- E poi si fa la densità indiretta
  Tutti i p che sono raggiungibili direttamente in maniera transitiva



  E anche qui non è riflessivo (siccome si inizia da un punto core)
  E da qui abbiamo creato dei primi cluster